

Decorators

 <https://github.com/learn-co-curriculum/python-p3-v2-decorators>  <https://github.com/learn-co-curriculum/python-p3-v2-decorators/issues/new>

Learning Goals

- Use decorators to add functionality to our functions while avoiding code redundancy.

Key Vocab

- **Decorator**: syntax that allows us to add functionality to an object without modifying its structure.

Introduction

Functions are reusable pieces of code that are called with a single expression. A function can be designed to accept arguments and return values, further increasing code reuse by adapting a computation based on actual values passed to arguments when a function is called.

A function follows an important software engineering principle of **Don't Repeat Yourself**. DRY is the principle of reducing repetition in our code by referencing a **single source** of reusable code whenever we need it.

Decorators provide us yet another way to write DRY code through providing extra functionality to functions beyond arguments.

Functions Are First Class Objects

While we primarily think of functions as procedures, it's important to remember that in Python, they are objects too. Just like any other object, they can be saved as variables, passed as arguments to other functions, and returned by functions:

```
def hello(name):  
    return "Hello " + name
```

```
print(hello("Guido"))  
# Hello Guido
```

```
greeting = hello  
print(greeting("Guido"))  
# Hello Guido
```

```
def salutation(func):  
    return func("Guido")
```

```
print(salutation(greeting))  
# Hello Guido.
```

Furthermore, we can define functions inside of other functions, just as we could any other object. We call these **inner functions**:

```
def hello(name):  
    print("Hello from the hello() function.")  
  
    def greet():  
        print("Greetings from the greet() function.")  
  
    return greet
```

What do you think we'll see if we `print()` the result of our `hello()` function?

```
hello("Guido")  
# Hello from the hello() function.  
# <function hello.<locals>.greet at 0x103287b80>
```

By returning `greet()` without parentheses, `hello()` is returning the function itself so that we can use it later on. When we're ready to invoke it later on, we can do so with parentheses as we would with any other function:

```
hello("Guido")()  
# Hello from the hello() function.  
# Greetings from the greet() function.
```

► What would be the output of the code above if `return greet` were left out?

Writing Your First Decorator

To write your first **decorator**, you'll need to tie all of these concepts together. You will need to write a function that...

1. Takes a function as an argument.
2. Has an inner function defined inside of it.
3. Returns the inner function.

Open up the Python shell and enter the following code:

```
def decorator(func):  
    def wrapper():  
        print("I am the output that lets you know the function is about to be called.")  
        func()  
        print("I am the output that lets you know the function has been called.")  
    return wrapper  
  
def get_called():  
    print("I am the function and I am being called.")
```

We've created a decorator and we've created a function to pass in. All that's left to do is put it all together:

```
get_called = decorator(get_called)
get_called()
# I am the output that lets you know the function is about to be called.
# I am the function and I am being called.
# I am the output that lets you know the function has been called.
```

Python allows us to perform the decoration step in a more decorative fashion with the `@` symbol. This is also called "*pie syntax*".

```
@decorator
def get_called():
    print("I am the function and I am being called.")

get_called()
# I am the output that lets you know the function is about to be called.
# I am the function and I am being called.
# I am the output that lets you know the function has been called.
```

When To Use Decorators

The primary function of decorators is reducing the amount of code that you need to write in your applications. If you find yourself reusing a lot of the same code in different functions, that's a great opportunity to use decorators. If you're only doing something once or twice, decorators might be overkill.

Let's look at an example of when we *would* want to use decorators.

```
def sweep_floors(time):
    if 1100 < time < 2100:
        print("Sweeping the floors...")
    else:
        print("I'm off duty!")
```

```
def wash_dishes(time):
    if 1100 < time < 2100:
        print("Washing the dishes...")
    else:
        print("I'm off duty!")

def chop_vegetables(time):
    if 1100 < time < 2100:
        print("Chopping the vegetables...")
    else:
        print("I'm off duty!")
```

There's a pretty clear pattern here: our employees only work from 11 to 9! Including code in every single function to check if anyone's working is not ideal. Let's refactor this with a decorator:

```
def check_working_hours(func):
    def wrapper(time):
        if 1100 < time < 2100:
            func(time)
        else:
            print("I'm off duty!")
    return wrapper

@check_working_hours
def sweep_floors(time):
    print("Sweeping the floors...")

@check_working_hours
def wash_dishes(time):
    print("Washing the dishes...")

@check_working_hours
```

```
def chop_vegetables(time):  
    print("Chopping the vegetables...")  
  
sweep_floors(800)  
# I'm off duty!  
wash_dishes(1000)  
# I'm off duty!  
chop_vegetables(1200)  
# Chopping the vegetables...
```

- What are the two options for invoking a decorator?

Conclusion

Functions are first-class objects in Python. This means that they can be passed as arguments to other functions, created inside of other functions (as *inner functions*), and returned by other functions. Decorators leverage these features to allow us to avoid repetitive code and are an important tool in any Python programmer's toolbox.

Resources

- [Python 3.8 Documentation](https://docs.python.org/3.8/)  (https://docs.python.org/3.8/)
- [Decorators in Python - GeeksforGeeks](https://www.geeksforgeeks.org/decorators-in-python/)  (https://www.geeksforgeeks.org/decorators-in-python/)